

Field Consistent Glyphs

David Hasse

Abstract—Visualizing fields is a crucial aspect of data visualization, yet traditional arrow glyphs and the multivariate nature of data often lead to misinterpretations, especially in curved fields. In this project, we develop a ParaView filter that constructs glyphs by integrating both parallel and orthogonal streamlines, as well as weighted combinations thereof, ensuring each glyph closely follows the underlying field. By preserving the field’s intrinsic characteristics—rather than relying on straight lines—our approach reduces ambiguity and enhances human interpretation of complex vector and tensor data. The resulting glyphs offer a more faithful depiction of flow behaviors, though they require additional computation compared to conventional arrow-based methods.

1 INTRODUCTION

Accurate visualization of vector and tensor fields is crucial across scientific and engineering disciplines—from fluid mechanics and meteorology to stress analysis in materials science. In many of these applications, arrow glyphs are the default choice for depicting local direction and magnitude: a straightforward arrow is placed at each sample point, oriented along the vector in question. While arrow glyphs are simple and intuitive, they are also inherently ambiguous in curved or complex fields. Specifically, a single arrow only precisely encodes the field at the point where it originates; elsewhere along its length, it can misrepresent changes in direction or magnitude. This shortcoming can lead to interpretive errors, particularly when the field exhibits strong curvature or contains critical points like sinks, sources, or saddle points.

The challenge becomes even more pronounced with tensor fields, which carry more multivariate information (e.g., anisotropy, principal directions) than a simple vector. Conventional glyphs struggle to capture these properties continuously and coherently. Consequently, researchers and practitioners risk missing subtle behaviors—like local rotations or uneven stress distributions—if the visualization method does not properly reflect the underlying data. These problems motivate the search for a new type of “field-consistent” glyph that remains faithful to the field at every point it occupies.

In this report, we focus on designing such field-consistent glyphs and implementing them, in part, as a ParaView filter. Our initial solution constructs glyphs by integrating along both parallel and orthogonal directions of the vector field, ensuring the glyphs geometry follows the local data rather than a simple straight line. Although we present a working filter, we also move beyond what is currently implemented to discuss alternative designs and future directions, including how field consistency might be extended to 3D domains, tensor fields, or even time-dependent visualization. By examining multiple approaches—some of which are only partially or conceptually explored—we highlight the potential for glyph-based methods that minimize ambiguity, preserve curvature, and offer clearer insight into the data. Ultimately, this work aims to lay a foundation for more advanced, flexible glyph representations that better convey the complexity of real-world fields.

2 RELATED WORK

Glyph-based visualization of vector and tensor fields has a long history, with researchers continually seeking designs that balance perceptual clarity, faithfulness to the underlying data, and control over visual clutter. This section first addresses the perceptual aspects of glyphs (including alternative designs and user studies), then discusses field consistency approaches that strive to maintain coherence with local data, and finally looks at strategies for clutter reduction. We close by placing our work within the broader field of scientific visualization.

2.1 Perceptual Effectiveness in Glyph Designs

Standard arrow glyphs remain popular due to their simplicity and familiarity, but a lot of research demonstrates that more sophisticated designs often improve interpretability, especially in complex or dynamic fields. For instance, Hlawatsch et al. [3] introduced flow radar glyphs that encode unsteady, time-dependent flow in a static 2D representation; by mapping time to radial distance and direction to angle, they allow viewers to grasp temporal and directional changes without animation. Although these glyphs are not “field-following,” they illustrate how altering glyph geometry can reduce cognitive strain by presenting multiple variables in a single, coherent shape.

In 3D settings, Stevens et al. [6] explored user performance with flow-field glyphs shaped as thin lines versus thicker tubes, showing that thicker, shaded tube glyphs often yielded higher accuracy for assessing flow orientation. They attributed this to stronger depth and shading cues, consistent with findings from Borgo et al. [1] that shape symmetry and proper lighting cues can significantly aid perception. Together, these studies highlight that shape, shading, and extension (e.g., curving the glyph to match local flow) can all improve a user’s ability to parse direction, magnitude, or other field properties at a glance.

A parallel line of research concerns tensor glyphs—for example, in diffusion MRI, ellipsoids or superquadrics are used to encode the principal directions and anisotropy of local diffusion tensors. These glyphs can be challenging to parse en masse, but Tong et al. [7] proposed an interactive “lens” that fades or hides occluding glyphs when the user zooms into regions of interest, preserving the 3D depth cues without overwhelming the viewer. Although this approach focuses on occlusion, it underscores that glyph design—and how it’s interactively managed—plays a central role in perceptual clarity.

2.2 Field Consistency and Coherent Mapping

Standard arrow glyphs accurately represent the vector direction only at the base of each arrow, leaving the rest of the shape detached from actual field values—an ambiguity that becomes pronounced in curved or rapidly changing fields. Hence, a central challenge is ensuring that each glyph (or each segment of a glyph) remains consistent with the underlying data it spans.

Some prior methods partially address this by curving glyphs along local streamlines, effectively turning each glyph into a short path that follows the field. Although Boring and Pang [2] still relied largely on straight arrows, they emphasized local shading improvements to clarify direction and vector size. In contrast, Hlawatsch et al. [3] introduced flow radar glyphs for time-dependent 2D fields, mapping flow direction to angle and time to radial distance—thus preserving temporal consistency in a single static glyph. This focus on consistent mapping is echoed in Peng and Laramee [5], who designed an algorithm to place glyphs at suitable intervals on complex CFD meshes, ensuring each glyph’s orientation truly follows the local flow.

Beyond vector fields, tensor visualization often uses ellipsoids or superquadric shapes whose axes align with the tensor’s eigenvectors—an approach that helps preserve local data structure. However, these shapes remain point-based (i.e., they don’t extend through space). Even so, they embody the principle of coherent mapping: at every point in the

display, the glyph is derived mathematically from the local data rather than being a generic arrow or sphere. Along similar lines, Wittenbrink et al. [8] tackled uncertainty visualization by varying glyph size and shape to show directional variance, thus maintaining fidelity to local data distributions.

Together, these methods confirm that field consistency – where glyph geometry (or orientation) directly encodes local flow or tensor properties – significantly reduces ambiguity. This work advances this idea by forming streamline segments that incorporate both parallel and orthogonal directions of the local field. Rather than relying on a single “arrow” shape at one point, each glyph becomes a small “field-driven structure,” ensuring it remains faithful to the data throughout its span.

2.3 Reducing Glyph Density and Visual Clutter

A hurdle in glyph-based visualization is clutter: too many glyphs can overlap or obscure key patterns, hindering interpretability. Researchers have proposed numerous strategies for mitigating clutter:

- **Selective Filtering:** Show glyphs only in regions of interest (e.g., high velocity or high anisotropy), omitting homogenous areas. For instance, in diffusion tensor MRI, ellipsoids are frequently restricted to areas exceeding a threshold.
- **Subsampling and Jittering:** Using a coarser sampling grid or randomly disrupting glyph positions can avoid overplotting while preserving overall trends. Laidlaw et al. [4] found that such jittering did not impair interpretive accuracy, but reduced distracting visual patterns.
- **Aggregation and Composite Glyphs:** Instead of drawing a separate glyph for every data point, polar histograms or “fan” glyphs can summarize local distribution, bundling multiple data vectors into one visual element.
- **Optimized Placement:** Algorithms that enforce minimal spacing prevent glyphs from bunching up. Peng and Laramée [5] used a method that places glyphs at regular intervals on complex meshes, ensuring coverage without excessive overlap.
- **Interactive Lensing:** Tong et al. [7]’s “glyph lens” hides or fades occluding glyphs as a user inspects particular regions, dynamically reducing clutter on demand.

These techniques are often complementary: topological or variance-based placement can be combined with interactive filtering, ensuring glyph-based visualizations remain comprehensible even at high data densities. For field-consistent glyphs (which may be visually more complex than simple arrows), these clutter-reduction ideas are particularly relevant, helping maintain a clear view of each glyph’s shape and trajectory.

2.4 Placement in the Scientific Context

Visualizing vector and tensor fields is central to many domains, including fluid mechanics, meteorology, materials science, and medical imaging. Over the years, a variety of glyph-based methods have surfaced to tackle domain-specific challenges – whether it is capturing time-varying flow structures, highlighting anisotropic diffusion, or reducing the clutter inherent in large-scale simulations.

Much of the earlier referenced work focused on either improving local fidelity (ensuring glyph orientation and magnitude match the data) or on managing the number of glyphs displayed. The interactive lens concept [7] demonstrates the additional value of real-time user control over how glyphs are rendered, pointing toward a trend of more flexible, user-steered visualization approaches.

Our work extends these foundations by modifying the shape of the glyphs themselves – curving them to follow both the parallel and orthogonal directions of the local field, thus maintaining consistency across the glyph’s entire geometry. In doing so, we build on streamline-based strategies but retain a recognizable “arrow” form. By integrating these ideas into ParaView, we aim to provide a practical filter that offers

a new degree of clarity in interpreting curved or complex fields. Future expansions of this method, such as full 3D glyph formation and adaptive placement guided by field topology, could push field-consistent glyphs even closer to the ideal of a continuous, unambiguous representation of scientific data.

3 FUNDAMENTALS

This section provides an overview of the essential mathematical and computational methods used in this project, which form the foundation for creating field-consistent glyphs.

Each subsection introduces a key method, explaining its function and relevance to this work.

3.1 Euler Method

The Euler Method is a simple numerical procedure for approximating solutions to ordinary differential equations (ODEs). In this project, it is used to trace streamlines in vector fields. By iteratively advancing in small steps along the vector direction, the method approximates the trajectory of a particle in the field. Its formula is:

$$\mathbf{p}_{n+1} = \mathbf{p}_n + h\mathbf{v}(\mathbf{p}_n) \quad (1)$$

where $\mathbf{p}_{n+1}$ is the next point, $\mathbf{p}_n$ is the current position, h the step size and $\mathbf{v}(\mathbf{p}_n)$ the vector field at $\mathbf{p}_n$.

3.2 Runge-Kutta 4 Method

Similarly to the Euler Method in Sect. 3.1, the Runge-Kutta 4 (RK4) Method is used to approximate solutions in ODEs. Through computing and weighing intermediate slopes, it achieves higher accuracy in its solutions than the aforementioned Euler Method. In this project, RK4 is used to provide an alternate, more accurate way to trace streamlines at the cost of a little longer computation time.

The weights for RK4 are computed as follows:

$$\begin{aligned} k_1 &= \mathbf{v}(\mathbf{p}_n) \\ k_2 &= \mathbf{v}\left(\mathbf{p}_n + \frac{k_1}{2}\right) \\ k_3 &= \mathbf{v}\left(\mathbf{p}_n + \frac{k_2}{2}\right) \\ k_4 &= \mathbf{v}(\mathbf{p}_n + k_3) \end{aligned} \quad (2)$$

The iteration formula is then defined as:

$$\mathbf{p}_{n+1} = \mathbf{p}_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4) \quad (3)$$

3.3 Bilinear Interpolation

In this project, Bilinear Interpolation is used to estimate the values of a vector field at a given point within a rectangular grid. This is crucial for accurately tracing streamlines in a vector field, as it allows for the interpolation of vector values at non-grid points. For a point (x, y) within a grid defined by (x_1, y_1) and (x_2, y_2) .

The interpolation formula is:

$$\begin{aligned} \mathbf{v}(x, y) &= q_{11} \cdot (1 - u_x) \cdot (1 - u_y) + q_{21} \cdot u_x \cdot (1 - u_y) \\ &+ q_{12} \cdot (1 - u_x) \cdot u_y + q_{22} \cdot u_x \cdot u_y \end{aligned} \quad (4)$$

where $u_x = \frac{(x-x_1)}{(x_2-x_1)}$, $u_y = \frac{(y-y_1)}{(y_2-y_1)}$ and $q_{ab} = \mathbf{v}(x_a, y_b)$ are the values of the vector field at the respective corners of the cell.

3.4 Bezier Splines

Bezier Splines are parametric curves used in computer graphics for modeling smooth and scalable curves. They are defined by using a set of control points $\mathbf{P}_0, \dots, \mathbf{P}_n$.

A cubic Bezier Spline, as used in this project, is given by:

$$\begin{aligned} B(t) &= (1-t)^3\mathbf{P}_0 + 3(1-t)^2t\mathbf{P}_1 \\ &+ 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3, \quad t \in [0, 1] \end{aligned} \quad (5)$$

This project utilizes Bezier Splines as a non-field-consistent means of forming a computationally heavy part of a glyph.

3.5 Bisection

The Bisection Method is a straightforward technique used to find an optimal solution within a given range. Starting with an interval $[a, b]$ where the solution is believed to exist, the method repeatedly halves this interval by setting a midpoint $c = \frac{a+b}{2}$. The interval is then updated by setting $a = c$ or $b = c$, depending on which half of the interval contains the solution.

3.6 Eigenvector Computation for Tensor Fields

In contrast to vector fields, tensor fields require additional processing to extract meaningful geometric representations. A tensor at any point in space is typically represented by a second-order matrix (e.g., a Jacobian or stress tensor) that describes variations in local quantities such as velocity, strain, or diffusion.

A symmetric second-order tensor T can be decomposed into its eigenvalues and eigenvectors, which define the principal axes of deformation or diffusion. The eigenvalue problem for a given tensor is formulated as:

$$T\mathbf{v} = \lambda\mathbf{v} \quad (6)$$

where T is the tensor matrix, λ is an eigenvalue (scalar), and $\mathbf{v}$ is the corresponding eigenvector (directional component).

3.6.1 Computing Eigenvectors

For a 3D tensor field, the matrix T is a 3×3 symmetric matrix, meaning it always has three real eigenvalues $\lambda_1, \lambda_2, \lambda_3$ and three corresponding orthogonal eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$. The eigenvectors are computed by solving the characteristic equation:

$$\det(T - \lambda I) = 0 \quad (7)$$

where I is the identity matrix.

For ellipsoidal tensor glyphs, the principal axes of the ellipsoid are aligned with the eigenvectors of the tensor matrix, and the scaling along each axis is proportional to the square root of the corresponding eigenvalue:

$$\text{Axis Length} \propto \sqrt{\lambda} \quad (8)$$

This ensures that the shape of the glyph accurately represents the local anisotropy of the field.

3.6.2 Application to Glyph construction

To construct glyphs from the previous eigenvalue and eigenvector information, we do the following:

1. **Major Axis Determination:** The largest eigenvector ($\mathbf{v}_1$) defines the primary direction of the tensor glyph.
2. **Minor Axes:** The remaining eigenvectors ($\mathbf{v}_2, \mathbf{v}_3$) define the secondary and tertiary axes, ensuring the glyph represents the full anisotropy of the field.
3. **Scaling:** The eigenvalues ($\lambda_1, \lambda_2, \lambda_3$) determine the length of each axis in the ellipsoidal glyph.

This approach enables the representation of tensor fields, preserving both directional and magnitude information in the visualization.

4 METHOD

In this project we worked with ParaView, an open source application for interactive scientific data analysis and visualization. As already mentioned in Sect. 1 and Sect. 2, we are pursuing the goal of developing filters for this application that create glyphs in such a way that they are completely consistent with the underlying field. For the implementation, we decided to use the Python programming language for both fields.

In the development process, ParaView ran in a virtual machine with Ubuntu 18.04.6 LTS as the operating system. In addition to the standard program, we used the so-called PRTL extension, which contains a series

of universal filters. Of these, we used the ‘Python Model’, which is available in both 2D and 3D, as the source for our fields, as it provides an easy way to test analytical functions on uniform grids. Why we need this is explained in more detail in Sect. 4.2.

In the following, we will explain in detail how these glyphs work for the different fields, how we built them, and generally explain the flow of these filters.

4.1 Tensor Field Glyphs

As already indicated in Sect. 2, we started the project with ellipsoidal tensors in three-dimensional space. Since the same problem arises here as with simple arrow glyphs – namely that it is not clear to which point along the glyph it applies – we wanted to achieve clarity through consistency with the field. To do this, we had to ask ourselves the fundamental question of how we could introduce this consistency into the glyphs. Our initial considerations were based on drawing an ellipsoidal tensor for an (arbitrary) starting point in the field. This starting point represents the “center point”, i.e. the intersection of the major and minor axes. The end points of the major axis of this glyph should in turn be starting points for two new ellipsoidal tensors, and so on. This would cause them to intersect and, we hypothesized, possibly converge to a larger, possibly curved ellipse that would represent the field in that area, see Fig. 1.

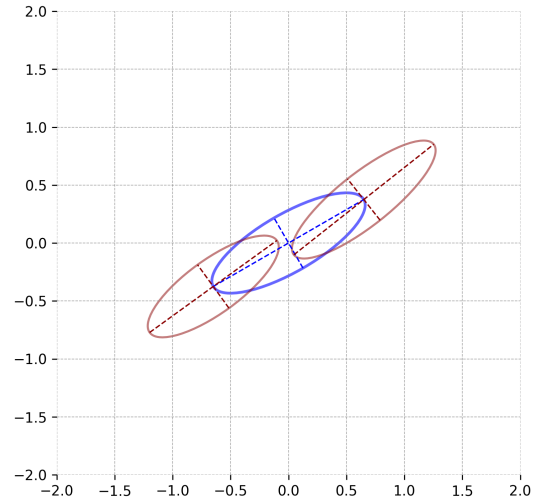


Fig. 1. Mock-Up for how these ellipsoidal tensor glyphs might look in a 2-Dimensional setting. The blue ellipsoid serves as an initial glyph, from which the final glyph grows by computing the major and minor axes of subsequent ellipsoids along the initial's major axis. In the final filter, only the outline of the whole glyph would be displayed.

The PRTL “Python Model 3D” source was used as it provides a uniform, rectilinear grid as well as the possibility to change the field via a script. Using this source as a base, we needed to construct a tensor field source. To create such a source, it is sufficient to assign a $n \times k$ matrix to each point of our grid. For this approach, we have used the Jacobian matrix to visualize the underlying field.

To construct our glyphs, we used the eigenvectors of the matrix at the respective point. The major axis is formed by the largest eigenvector, while the minor axis is constructed by the second largest eigenvector and – since we have considered the 3D case – the third eigenvector determines the depth axis.

From an arbitrarily chosen starting point in the grid, we construct the glyphs by following the major axis for each point, branching outwards according to the minor axes for every step. To do this, we look at the

eigenvectors at the current data point and follow the normalized eigenvector with the largest eigenvalue in order to illustrate the dynamics of the field. If we reach the boundary of the space or field we are looking at, we stop, clipping everything outside of the space.

4.1.1 Glyph Variants

To allow flexible visualization of different field properties, we implemented several glyph variants, selectable via a slider in ParaView. These variants differ in how they incorporate the largest, second-largest, and third-largest eigenvectors of the underlying tensor field.

Each Variant follows a consistent procedural approach:

1. **Variant 0 (Baseline)** – Constructs a streamline following the largest eigenvector from a starting point, extending it iteratively until reaching field boundaries.
2. **Variant 1** – Extends Variant 0 by additionally incorporating the second-largest eigenvector, following its natural direction from each data point.
3. **Variant 2** – Similar to Variant 1, but instead of following the second-largest eigenvector in a single direction, it is symmetrically averaged, forming a more ellipse-like shape.
4. **Variant 3** – Extends Variant 2 by adding the third-largest eigenvector, following its natural direction.
5. **Variant 4** – Similar to Variant 3, but the third-largest eigenvector is symmetrically averaged, mirroring the approach of the second-largest eigenvector to form a 3D elliptical glyph.
6. **Variant 5** – Incorporates all three eigenvectors, following each in its natural orientation.

These variants were designed to explore whether field-consistent glyphs could converge into a larger, superordinate geometric form – ideally an ellipse – when overlaid in a structured manner. However, no clear convergence was observed, as each glyph would grow until either the whole space was filled or a boundary was reached. Efforts to prevent this behavior could include the usage of a decreasing factor, scaling the eigenvector’s normalized length down for each step. This would, however, introduce an arbitrary scalar factor and potentially misrepresent the underlying field, defeating the point of this approach to create “field-consistent” glyphs. Due to the complexity of tensor-based glyph visualization in three-dimensional space, we shifted our primary focus to vector glyphs in two-dimensional space, where the core question of “field consistency” is easier to illustrate and analyze.

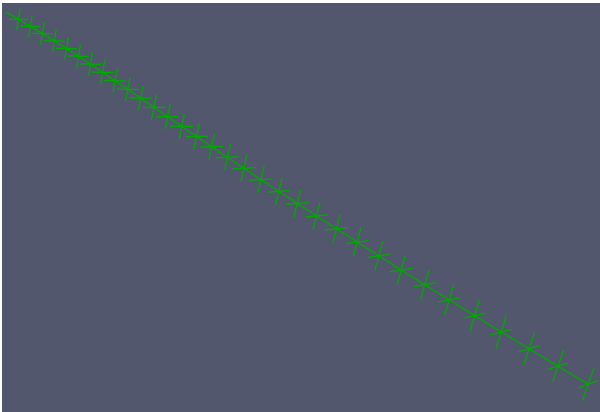


Fig. 2. Visualization of the “skeleton” of a tensor glyph. Following the largest eigenvector and fanning out according to the second and third-largest eigenvectors. A completed version of this glyph would incorporate a elliptical hull for this skeleton.

4.2 Vector Field Glyphs

In many scientific and engineering applications, vector fields in 2D must be visualized in a manner that clearly conveys both local direction and curvature. Although simple arrow glyphs are commonplace, they often fail to represent changes in the field along the length of each glyph. To address this, our approach constructs “field-consistent” arrow glyphs by tracing local streamlines and by optionally sampling orthogonal directions to create well-defined arrow shafts and heads. The result is a more faithful depiction of continuous field behavior – especially in curved or vortex-prone regions – than is offered by conventional straight arrow glyphs.

4.2.1 Overview of the Construction Pipeline

1. **Sampling Locations:** We begin by placing glyph “seed points” at each cell of an $n \times m$ grid; optionally, users can specify an additional point for which to create a glyph. Each seed point (x_i, y_i) becomes the center of a new arrow glyph.
2. **Main Streamline:**
 - From each seed point, we trace the parallel direction of the field forward and backward, each for a user-defined number of steps $N_{\text{length}}/2$.
 - Each step uses either Euler or RK4 integration (both are supported and user-selectable). Users can also enable or disable vector normalization.
 - The result is a smooth, curved center line (the arrow “spine”) with the seed point at its midpoint. We refer to this as the main streamline.
3. **Arrow Base:**
 - At the backward end of the main streamline, we sample the field’s orthogonal direction (both orientations) for a specified number of steps N_{width} .
 - These two short orthogonal traces define the “shaft width” at the arrow’s base.
4. **Secondary Streamlines:**
 - From each endpoint of the arrow base, we again trace forward along the parallel direction for $N_{\text{length}}/2$ steps.
 - This produces two curved lines that run alongside the main streamline, collectively forming a “shaft” region.
5. **Arrowhead Base and Tip:**
 - We form the arrowhead base by again sampling the orthogonal direction at the shaft’s side’s forward end.
 - To close the arrowhead and reach the main streamline’s forward tip, we employ one of several arrowhead variants (see Sect. 4.2.3). Each variant handles the transitions from the sides of the arrow shaft to the forwardmost point on the main streamline in a distinct way.

4.2.2 Key Parameters

Our filter provides various adjustable parameters for controlling glyph size, density, and integration details. The most important include:

- **Grid Dimensions (n, m) :** Determines how many glyphs are placed across the domain. A larger grid produces more, potentially overlapping glyphs, choosing appropriate grid dimensions is essential to avoid clutter.
- **Seed Point:** One can specify a custom seed point if a local region of interest requires detailed inspection

- **Number of Steps:**

- N_{length} Total length of the glyph in units (half forward, half backward).
- N_{width} Total width of the arrow base and arrowhead width.

- **Stepsize (h):** The distance advanced at each integration step. Larger h yields a coarser approximation but faster execution; smaller h captures fine curvature details at higher computational cost.
- **Normalization:** When on, every vector is normalized to unit length before each step, creating uniformly sized arrows. When off, arrows reflect the actual vector magnitudes in the field, but may vary greatly in size.
- **Scaling Factor (s):** A factor scaling the length of every vector during computation. Can be used to overall scale glyph size down without the need to normalize.
- **Integration Method:** A toggle between Euler (faster, less accurate) and RK4 (slower, more accurate).
- **Arrowhead Shape:** A toggle between the different shapes of the arrowhead explained in Sect. 4.2.3.

All parameters can be fine-tuned through the ParaView interface, allowing interactive experimentation based on the user's data or desired visual fidelity.

4.2.3 Arrowhead Variants

We provide three different strategies for closing the arrowhead from its base to the tip of the main streamline:

1. **Smooth Arrowhead (Weighted Combination)**

Idea: At each small step along the arrowhead, blend the parallel and orthogonal field directions:

$$\Delta \mathbf{p} = a \mathbf{v}_{\parallel} + b \mathbf{v}_{\perp}, \text{ where } a + b = 1. \quad (9)$$

Implementation: We solve for appropriate a and b values using a bisection-like approach so that the resulting streamline eventually meets the arrow's front tip.

When to Use: In gently curving fields where a smooth, consistently field-following arrowhead is desired.

2. **Staircase Arrowhead**

Idea: Alternate short steps in the parallel direction with short steps in the orthogonal direction, resulting in a "zigzag" path that eventually converges on the forward tip.

Characteristics: The shape appears jagged, which is further accentuated in regions of high curvature and is more stable in stable fields.

When to Use: When a purely continuous approach struggles to converge or yields unappealing arrowhead geometry, the simple stepwise alternation is often more robust and efficient.

3. **Interpolated Arrowhead (Bezier Splines):**

Idea: Directly connect the arrowhead base endpoints to the main streamline tip using Bezier curves.

Tradeoff: This breaks strict field consistency in the arrowhead region, as only the control points are computed using vector field information, but ensures a connected tip.

When to Use: If the user values computation time more than strict field consistency for the tip, especially in high-vorticity regions where purely field-following approaches might twist undesirably and fail to converge.

These tip variants are illustrated in Fig. 4, Fig. 6, and Fig. 8 for the same seed points in some sample fields.

4.2.4 Implementation Notes and Practical Tips

Performance: Because each arrow glyph traces multiple streamlines, runtime grows with the number of seed points and the chosen step counts. To avoid slow computation, shrinking the $n \times m$ grid or choosing a higher step size may be necessary.

Edge Cases: In high-curvature fields or near singularities (sinks, sources, vortices), arrow shapes can become very large or self-intersecting. One solution would be to use adaptive step sizes that shrink in areas of high curvature or to terminate an arrow if it re-enters a previously visited region.

Clutter Reduction: When vectors have large magnitudes, unnormalized arrows can dominate the display. Consider either normalizing vector, scaling the overall glyph size down, or applying fewer seed points to avoid excessive overlap.

User Interaction: In ParaView, interactive sliders allow quick tuning of parameters. Observing how arrows change with different step sizes or arrowhead variants often reveals the field's structure more clearly than static glyphs alone.

5 RESULTS

In this section, we present the outcomes of our field-consistent glyph approach on a variety of 2D vector fields with different characteristics (e.g., radial, rotational, and saddle-like fields). We compare the resulting glyphs to standard arrow glyphs to highlight differences in interpretability, and we comment on the computational aspects that arise with more complex fields. Finally, we conclude with a discussion of key observations and trade-offs.

5.1 Outputs

To demonstrate our methods, we tested three types of synthetic vector fields, each representing a different flow characteristic:

1. **Radial Field:** $F_1(x, y) = (x, y)^T$, which expands outward from the origin.
2. **Saddle Field:** $F_2(x, y) = (x, -y)^T$, featuring a saddle-like behavior.
3. **Rotational Field:** $F_3(x, y) = (-y, x)^T$, producing a constant angular velocity around the origin.

Below, we show representative images generated by our ParaView filter for each field, comparing Smooth Arrowhead, Staircase Arrowhead, and Bezier Interpolation arrow tips.

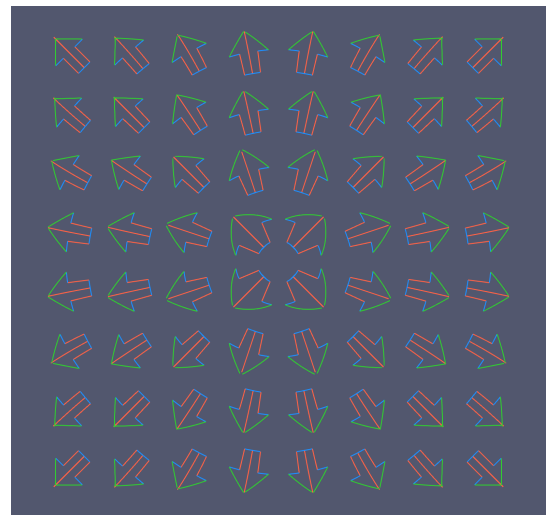


Fig. 3. Radial Field $F = (x, y)^T$. Smooth Arrowhead variant. Notice that each glyph smoothly curves outward, faithfully reflecting how the field radiates from the origin.

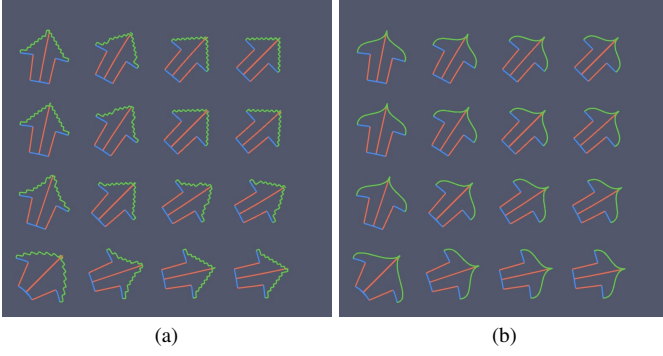


Fig. 4. Radial Field $F = (x, y)^T$. (a) Staircase Arrowhead, (b) Bezier Arrowhead. Notice how (a) similar to the Smooth variant, albeit lower resolution, the jagged arrow tips look in low variant areas. (b) produces a not field-following tip shape, which can still be used to interpret the field semi-accurately.

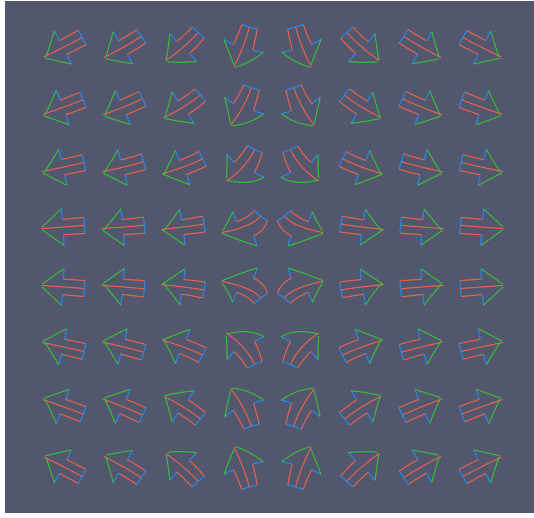


Fig. 5. Saddle Field $(x, -y)^T$. Smooth Arrowhead variant. The arrows bend inward and outward across the saddle point at $(0, 0)$.

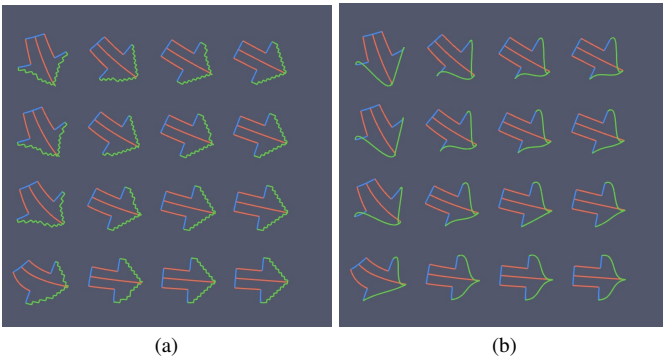


Fig. 6. Saddle Field $(x, -y)^T$. (a) Staircase Arrowhead, (b) Bezier Arrowhead.

5.2 Interpretation / Comparison

Our "field-consistent" glyphs aim to resolve a fundamental ambiguity in conventional arrow glyphs: the straight-line arrow encodes a vector only at its base, leaving the rest of the arrow susceptible to misinterpretation in curved or rapidly changing fields. By contrast, our glyphs follow streamlines derived from the local vector field – both in the primary (parallel) and secondary (orthogonal) directions – thus conveying the flow more continuously along the glyphs entire length. In practice, this

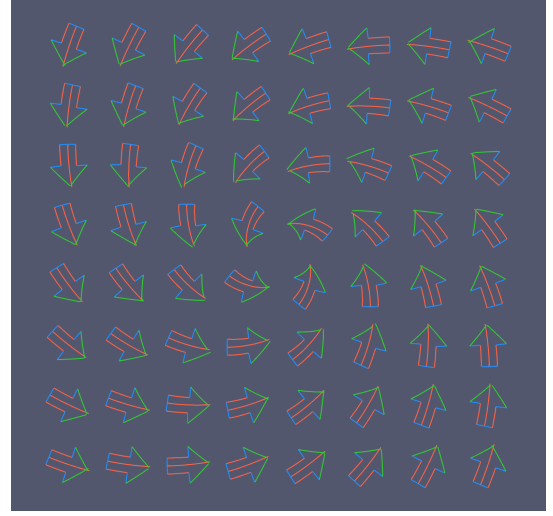


Fig. 7. Saddle Field $(-y, x)^T$. Smooth Arrowhead variant. The arrows form concentric swirls around the origin, clearly illustrating the rotational nature.

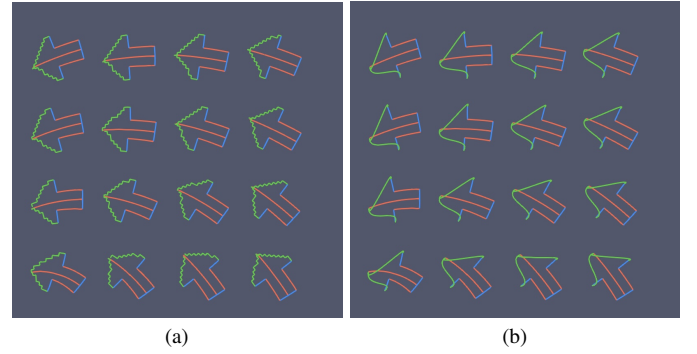


Fig. 8. Rotational Field $(-y, x)^T$. (a) Staircase Arrowhead, (b) Bezier Arrowhead.

design choice tends to reduce visual ambiguity, as each segment of the glyph is directly informed by the underlying vectors.

5.2.1 Fewer, More Informative Glyphs

One particularly noteworthy benefit is that fewer "field-consistent" glyphs can suffice to convey important structures in the data, relative to traditional arrows. Simple arrow glyphs require very high density – sometimes thousands of arrows – to depict the same sense of curvature or directional change, especially if the field exhibits strong vortices or variable directions. As a result, traditional arrow glyphs become more easily cluttered or confusing unless they are carefully subsampled or filtered. By contrast, our method's glyphs are richer in detail, capturing local field variations within each arrow's geometry. This allows for a lower seeding density in many cases, which can help prevent cluttering the screen while still giving a clear picture of the flow.

5.2.2 Higher Computational Cost

However, the previous advantage arises partly out of necessity: our glyphs are more computationally expensive to generate. Rather than simply placing a straight arrow based on a single vector at one point, our method integrates along parallel and orthogonal directions, often employing higher-order numerical approaches (RK4) or interpolation for arrowhead construction. As a result, we can only generate one of our glyphs in the same time it would take to place potentially thousands of normal arrows. From a performance standpoint, this tradeoff means we must be strategic in the number of glyphs we seed. In many applications – especially in large data sets – the cost factor is non-trivial and must be

balanced against the clarity gains provided by our glyphs.

5.2.3 Balancing Density and Complexity

In workflows where interactive exploration is key, standard arrow glyphs still have their place: they can be generated quickly, and a user can rapidly scan an overview of the entire field. Our approach meanwhile suits scenarios where accuracy of local flow is important, the data exhibits more complex features, or data visualization does not need to be in real time (e.g. generating an appropriate grid of our glyphs to later present for, e.g., teaching scenarios). Further integration into user-friendly environment – such as automated seeding based on field topology or adaptive density near critical regions – could help mitigate the computational drawbacks.

5.3 Future Work

The “field-consistent” glyphs presented in this report already show how a field-consistent visualization of vector fields can be realized in 2D (and, to a limited extent, in 3D for tensor fields). Nevertheless, a number of additional questions and extensions open up, which are outlined below. These proposals include both methodological and algorithmic approaches as well as aspects of practical implementation and evaluation.

5.3.1 Topologically Controlled Glyph Placement

Up to now, glyphs have usually been placed on equidistant grid points or via user-defined starting points. However, such uniform placement can become inefficient or confusing if the field has very different characteristics (e.g. vortex regions versus uniform regions). By integrating topological concepts into the placement algorithm, a more meaningful and “tidy” visualization can be achieved.

This would involve the following steps:

- **Extraction of critical points**
A topological analysis is carried out using established methods (e.g. eigenvalue analyses of the Jacobian matrix, index calculations). Critical points (sources, sinks, saddles) and separating lines are identified in order to segment the field into sub-areas.
- **Adaptive glyph density**
A higher density of smaller glyphs is placed in regions of high curvature or near critical points. In contrast, fewer, bigger, more widely spaced glyphs are used in approximately homogeneous areas. A possible algorithm could use the local field variance as an indicator: $\sigma(x,y)$ measures how much the vectors change in a small neighborhood. Where σ is high, the starting points are placed closer together.
- **Evaluation criteria**
Visual clarity: Reduction of cluttering through adaptive distancing. Information content: Improvement of the recognizability of special topological structures.

This approach means that the field consistency of the glyphs is maintained, but at the same time, intelligent placement simplifies the resulting image and emphasizes the essential field structures.

5.3.2 3D Vector Field Variant

One option that remains open is the transfer to three-dimensional vector fields. We had already discussed this variant, but did not manage to implement it.

Although the methods presented in 2D already offer added value in terms of improved interpretability and representation of the underlying field, many real-world applications (e.g. flow simulations, meteorological data) are intrinsically three-dimensional.

Extending our field-consistent glyph method into 3D would be a natural evolution, preserving many aspects of arrow construction, possibly by placing “arrow tubes” or full 3D streamline-based glyphs around a central path. This raises new questions, including how to manage occlusion and overlap, how to represent arrowheads in full 3D, and how to maintain clarity in highly curved or twisting flows. Incorporating true

3D interpolation methods (e.g., trilinear or higher-order approaches) and possibly leveraging adaptive seeding or particle tracing techniques could also help manage the increased complexity. Moreover, for fields defined on curved surfaces, one must ensure glyphs remain tangential to the manifold rather than floating arbitrarily in space.

5.3.3 Time Dependency and Uncertainty

In many domains, the vector field is not static but time-dependent. With unsteady fields, the direction and magnitude change over time. Instead of visualizing just one time step, we could:

- **Animate glyphs**
Each glyph is recalculated and dynamically updated for each time step. This enables a continuous “flow” in the field, but increases the computational and visualization requirements even more.
- **Time-compressed representations**
Methods such as “Flow Radar Glyphs” (cf. Hlawatsch et al. [3]) could be combined with the field consistency approach to accommodate several time steps in a static visualization.

Since vector fields in real simulation and measurement data are often subject to uncertainties (measurement noise, simulation inaccuracies), the following could be undertaken as future work to further improve their visualization:

- **Probabilistic glyphs**
Instead of a deterministic field, an ensemble of fields is used. The main streamline represents the expected value, while conical “variance envelopes” (e.g. uncertainty bands) indicate deviations.
- **Colour coding**
An additional, alpha-transparent layer along the arrow curve could signal the deviation of the individual simulations from the mean value.

These methods would further increase the informative value of field-consistent glyphs by integrating variability or confidence intervals into the image.

5.3.4 User Interaction and Evaluation

To make field-consistent glyphs more practical in day-to-day analysis, further work could incorporate interactive manipulation and real-time parameter tuning. Ideally, users should be able to relocate or remove glyph seeds, change integration step sizes, or dynamically adjust arrow-head variants with immediate visual feedback. This form of interactive “glyph lens” would allow domain experts to focus on regions of interest without cluttering the entire field. Additionally, there is a need for systematic user studies to evaluate how well different glyph styles – smooth vs. staircase arrowheads, for instance – improve a viewer’s accuracy, speed, and confidence in identifying field structures. Controlled experiments could measure whether field-consistent glyphs outperform standard arrow glyphs on tasks like detecting critical points or estimating curvature, which would further validate or refine our approach.

5.3.5 Addressing Current Limitations

Finally, some existing limitations in the method itself warrant further attention. Our 2D tensor approach remains preliminary, and it is unclear whether curved tensor glyphs can converge into larger coherent shapes in a way that consistently represents all local eigenvalues or tensor directions without confusion. Additionally, the computational overhead of tracing multiple streamlines per glyph could be addressed through more efficient integration algorithms or parallelization on modern hardware. By balancing the cost of glyph generation against the information gained, future work can refine the sweet spot between performance and visual clarity. Extending the streamline approach to incorporate localized decimation, load balancing across multiple processing units, or GPU-accelerated integration are all feasible strategies for large-scale data sets.

In summary, there is ample scope for improving and broadening the application of field-consistent glyphs. Whether through smarter

placement informed by topology, extension into three-dimensional or time-varying domains, improved uncertainty encoding, or deeper user evaluation, each of these directions promises to enhance our ability to visualize and interpret complex vector data. By continuing to innovate in these areas, researchers and practitioners can push field-consistent glyphs closer to their ideal: a cohesive, unambiguous, and computationally tractable representation of vector and tensor fields in real-world scenarios.

6 CONCLUSION

In this report of a software practical, we introduced a field-consistent glyph approach for visualizing vector fields (and, to a limited extent, tensor fields) within ParaView. Our central motivation was to reduce the ambiguity inherent in standard arrow glyphs, which only strictly encode the vector at their base. By constructing glyphs from both parallel and orthogonal streamline segments – and merging them into familiar arrow-like shapes – we demonstrated that it is possible to represent the local flow more faithfully along each glyph’s entire length. Comparative evaluations against traditional arrow glyphs showed that our method can reveal subtle curvature and flow behaviors with fewer glyphs, at the cost of greater computational overhead per glyph.

Although our results underscore the promise of field-consistent glyphs in clarifying curved or rapidly varying fields, several challenges and open questions remain. Most notably, the computation time for streamline integration may become expensive for large datasets or interactive scenarios, pointing to the need for adaptive seeding, parallelization, or on-demand glyph generation. Additionally, while our preliminary work on 3D tensor glyphs suggests the possibility of extending the same principles into higher dimensions, a host of occlusion and complexity issues emerges when moving beyond two-dimensional domains.

Despite these limitations, our method succeeds in highlighting the potential for a more nuanced and continuous representation of vector fields. By capturing not just the direction at a single point, but also the path that the flow takes between neighboring points, these glyphs reduce interpretive errors and provide a richer depiction of the underlying data. Building on this foundation, future efforts can refine glyph placement (for instance, via topological analysis), extend to fully three-dimensional contexts, account for time-dependent flows, and incorporate uncertainty measures. We believe that continuing to explore these directions will lead to increasingly robust, flexible, and informative approaches for visualizing complex vector and tensor fields across a wide array of scientific and engineering applications.

REFERENCES

- [1] R. Borgo, J. Kehler, D. H. S. Chung, E. Maguire, R. S. Laramée, H. Hauser, M. Ward, and M. Chen. Glyph-based Visualization: Foundations, Design Guidelines, Techniques and Applications. In M. Sbert and L. Szirmay-Kalos, eds., *Eurographics 2013 - State of the Art Reports*. The Eurographics Association, 2013. doi: /10.2312/conf/EG2013/stars/039-063
- [2] E. Boring and A. Pang. Directional flow visualization of vector fields. In *Proceedings of Seventh Annual IEEE Visualization '96*, pp. 389–392, 1996. doi: 10.1109/VISUAL.1996.568139
- [3] M. Hlawatsch, P. Leube, W. Nowak, and D. Weiskopf. Flow radar glyphs—static visualization of unsteady flow with uncertainty. *IEEE Transactions on Visualization and Computer Graphics*, 17:1949–58, 12 2011. doi: 10.1109/TVCG.2011.203
- [4] D. Laidlaw, E. Ahrens, D. Kremers, M. Avalos, R. Jacobs, and C. Readhead. Visualizing diffusion tensor images of the mouse spinal cord. In *Proceedings Visualization '98 (Cat. No.98CB36276)*, pp. 127–134, 1998. doi: 10.1109/VISUAL.1998.745294
- [5] Z. Peng, R. S. Laramée, O. Deussen, D. Keim, and D. Saupe. Vector glyphs for surfaces: A fast and simple glyph placement algorithm for adaptive resolution meshes. In *VMV*, pp. 61–70, 2008.
- [6] A. H. Stevens, T. Butkiewicz, and C. Ware. Hairy slices: Evaluating the perceptual effectiveness of cutting plane glyphs for 3d vector fields. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):990–999, 2017. doi: 10.1109/TVCG.2016.2598448
- [7] X. Tong, C. Li, and H.-W. Shen. Glyphlens: View-dependent occlusion management in the interactive glyph visualization. *IEEE Transactions*

on Visualization and Computer Graphics, 23(1):891–900, 2017. doi: 10.1109/TVCG.2016.2599049

- [8] C. Wittenbrink, A. Pang, and S. Lodha. Glyphs for visualizing uncertainty in vector fields. *IEEE Transactions on Visualization and Computer Graphics*, 2(3):266–279, 1996. doi: 10.1109/2945.537309